

Note: Write your code in the code cells, and your responses in markdown. Run the entire script and display the outputs of your code.

Due: **11:59PM Central Time on Monday, 11/03**. Upload both your code (.ipynb) and responses (html or pdf) to Canvas by then.

AI Assistance Disclaimer

I used AI tools to help troubleshoot minor coding issues and reorganize sections of my notebook for clarity. Specifically, AI assistance was used to improve code readability (e.g., spacing, indentation, and commenting) and ensure that my workflow and outputs were well documented and easy to follow. All data analysis, interpretation, and written responses are my own.

```
In [1]: # --- PS5 setup & data load (robust paths, env check) ---
import sys, os
from pathlib import Path
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf # <- formulas

pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

print("Python:", sys.version.split()[0])
print("Kernel executable:", sys.executable)
print("CWD:", Path.cwd())

# Resolve project root (works in VS Code without manual `cd`)
nb_dir = Path.cwd()
project_root = nb_dir

# Candidate locations for the CSV
candidates = [
    project_root / "twins.csv",           # same folder as notebook
    project_root / "Input" / "twins.csv", # PS5/Input next to notebook
    project_root.parent / "Input" / "twins.csv", # notebook in PS5/, default
    project_root / ".." / "Input" / "twins.csv", # safety for odd launches
]
candidates = [p.resolve() for p in candidates]

csv_path = next((p for p in candidates if p.exists()), None)
if csv_path is None:
    raise FileNotFoundError(
        "Could not find twins.csv.\nTried:\n  - " + "\n  - ".join(map(str, candidates))
    )

print("Using data file:", csv_path)

twins = pd.read_csv(csv_path)
```

```
print("Rows, Cols:", twins.shape)
display(twins.head())
```

Python: 3.11.14

Kernel executable: /opt/anaconda3/envs/econ695/bin/python

CWD: /Users/alexdelatorre/Documents/School/UW-Madison/FALL 2025/ECON 695/Problem Sets/PS5/Input

Using data file: /Users/alexdelatorre/Documents/School/UW-Madison/FALL 2025/ECON 695/Problem Sets/PS5/Input/twins.csv

Rows, Cols: (1074, 13)

	famid	age	daded_a	t	lw	married	omarried	educ	oeduc	female	c
0	2	33.2512	12.0000	1	2.4795	0	0	16.0000	16.0000	1	
1	2	33.2512	12.0000	2	2.2202	0	0	16.0000	16.0000	1	
2	4	43.5701	12.0000	1	2.9495	0	1	18.0000	12.0000	1	
3	4	43.5701	12.0000	2	2.2282	1	0	12.0000	18.0000	1	
4	6	30.9678	12.0000	1	2.8509	1	1	12.0000	12.0000	0	

NOTE from 10/29

- i = family
- t = twin

Princeton Twins Data

In this problem set we will fit a few models to the Princeton Twins Survey data. The data set is called twins.csv. The variables are:

- famid = family id variable
- t=1,2 for twin #1 or twin #2
- age = age (some observations have coded in part year values)
- educ = education
- oeduc = education of twin
- lw = log wage
- married = dummy 1 if married 0 if not
- omarried = dummy if twin is married
- female = 1 if female
- ofemale = 1 if twin is female.
- exp = "labor market experience" = age - educ - 6
- oexp = experience of twin

NOTE: all twins are one of two identical twins in the data set. So female=ofemale in all cases.

```
In [13]: # Load dataset (using pandas "pd")
twins = pd.read_csv("twins.csv") # add your own directory if necessary
print(twins.head(6))

# verify the data is unique at (famid, t) level, where t = 1,2 for twin #1 c
assert 0==twins[['famid','t']].duplicated().sum()

# verify twins have the same sex:
assert (twins['female']==twins['ofemale']).all()
```

	famid	age	daded_a	t	lw	married	omarried	educ	oeduc	\
0	2	33.25119	12.0	1	2.479523	0	0	16.0	16.0	
1	2	33.25119	12.0	2	2.220177	0	0	16.0	16.0	
2	4	43.57014	12.0	1	2.949527	0	1	18.0	12.0	
3	4	43.57014	12.0	2	2.228209	1	0	12.0	18.0	
4	6	30.96782	12.0	1	2.850934	1	1	12.0	12.0	
5	6	30.96782	12.0	2	2.862515	1	1	12.0	12.0	

	female	ofemale	exp	oexp
0	1	1	11.25119	11.25119
1	1	1	11.25119	11.25119
2	1	1	19.57014	25.57014
3	1	1	25.57014	19.57014
4	0	0	12.96782	12.96782
5	0	0	12.96782	12.96782

Before that going into Q1, we want to just do a few data checks detailed in the following cell.

```
In [2]: # --- Features & quick data checks ---
twins = twins.copy()

# experience^2
twins["exp2"] = twins["exp"]**2

# quick sanity checks used later
assert twins[["famid","t"]].duplicated().sum()==0, "famid,t should be unique
assert (twins["female"]==twins["ofemale"]).all(), "All pairs are same-sex by

twins[["lw","educ","exp","exp2","married","female"]].describe().T
```

Out [2]:

	count	mean	std	min	25%	50%	75%	
lw	1,074.0000	2.4710	0.6098	0.7226	2.0441	2.4571	2.8212	4.8
educ	1,074.0000	14.0805	2.0298	8.0000	12.0000	14.0000	16.0000	18.0
exp	1,074.0000	17.8336	11.8061	-2.0000	9.0000	16.7361	25.0000	53.0
exp2	1,074.0000	457.2928	518.2904	0.0000	81.0000	280.1114	625.0000	2,809.0
married	1,074.0000	0.4795	0.4998	0.0000	0.0000	0.0000	1.0000	1.0
female	1,074.0000	0.5345	0.4990	0.0000	0.0000	1.0000	1.0000	1.0

1. OLS

Estimate a simple model relating log wages to: education, experience, experience-squared, married status, and female.

In [3]:

```
# --- OLS with HC1 robust SEs ---
import statsmodels.formula.api as smf
import pandas as pd

formula_q1 = "lw ~ educ + exp + exp2 + married + female"
ols_q1 = smf.ols(formula_q1, data=twins).fit(cov_type="HC1")

# Tidy table (coef, robust se, t, p, N, R2)
tbl_q1 = (
    pd.DataFrame({
        "coef": ols_q1.params,
        "se(HC1)": ols_q1.bse,
        "t": ols_q1.tvalues,
        "p>|t|": ols_q1.pvalues
    })
    .round(4)
)
display(tbl_q1)
print(f"N = {int(ols_q1.nobs)}, R2 = {ols_q1.rsquared:.4f}, R2_adj = {ols_q1
```

	coef	se(HC1)	t	p> t
Intercept	0.3455	0.1386	2.4934	0.0127
educ	0.1238	0.0086	14.3617	0.0000
exp	0.0523	0.0048	10.8826	0.0000
exp2	-0.0009	0.0001	-8.2757	0.0000
married	0.0693	0.0321	2.1568	0.0310
female	-0.3341	0.0309	-10.8267	0.0000

N = 1074, R2 = 0.3146, R2_adj = 0.3113

Q1. Baseline OLS Regression: Log Wages and Individual Characteristics

We estimate the model

$$\ln w_i = \beta_0 + \beta_1 \text{educ}_i + \beta_2 \text{exp}_i + \beta_3 \text{exp}_i^2 + \beta_4 \text{married}_i + \beta_5 \text{female}_i + u_i$$

using heteroskedasticity-robust (HC1) standard errors.

Variable	Coefficient	Robust SE	t	p
Intercept	0.3455	0.1386	2.49	0.013
Education	0.1238	0.0086	14.36	0.000
Experience	0.0523	0.0048	10.88	0.000
Experience ²	-0.0009	0.0001	-8.28	0.000
Married	0.0693	0.0321	2.16	0.031
Female	-0.3341	0.0309	-10.83	0.000
N = 1,074 R ² = 0.3146 R ² _adj = 0.3113				

Interpretation.

Education has a strong, positive, and statistically significant effect on log wages: each additional year of schooling raises expected log wages by about 0.124, or roughly 12.4%.

Experience also increases wages but at a diminishing rate: the positive coefficient on exp (0.052) and negative coefficient on exp^2 (-0.0009) imply a concave experience-wage profile. The implied turning point is approximately:

$$\text{exp}^* = \frac{0.0523}{2(0.0009)} \approx 29 \text{ years}$$

beyond which additional experience yields little wage growth.

Being married is associated with about a 6.9% higher wage, statistically significant at 5%. In contrast, the coefficient on female (-0.334) indicates that, holding education and experience constant, women earn approximately 33% lower wages than men on average.

The model explains about 31% of the variation in log wages ($R^2 = 0.3146$), consistent with a well-fitting cross-sectional wage regression.

2. Separate models for men and women

Fit the same model separately for men and women. Does the model look different?

- Note: drop "female" from the regression because we are estimating the regression separately by gender.

```
In [5]: # --- Q2: Separate OLS regressions for men and women (HC1 robust) ---

formula_q2 = "lw ~ educ + exp + exp2 + married"

# Separate by gender
men = twins.loc[twins["female"] == 0]
women = twins.loc[twins["female"] == 1]

# Fit OLS models with robust SEs
ols_men = smf.ols(formula_q2, data=men).fit(cov_type="HC1")
ols_women = smf.ols(formula_q2, data=women).fit(cov_type="HC1")

# Tidy summary table function
def tidy_ols(result, label):
    import pandas as pd
    tbl = (
        pd.DataFrame({
            "coef": result.params,
            "se(HC1)": result.bse,
            "t": result.tvalues,
            "p>|t|": result.pvalues
        })
        .round(4)
    )
    print(f"{label} results - N={int(result.nobs)}, R²={result.rsquared:.4f}")
    display(tbl)
    return tbl

# Display results
tbl_m = tidy_ols(ols_men, "Men")
tbl_f = tidy_ols(ols_women, "Women")
```

Men results - N=500, $R^2=0.2553$, $R^2_{adj}=0.2493$

	coef	se(HC1)	t	p> t
Intercept	0.5197	0.2194	2.3685	0.0179
educ	0.1027	0.0135	7.5896	0.0000
exp	0.0588	0.0080	7.3951	0.0000
exp2	-0.0010	0.0002	-5.8950	0.0000
married	0.1993	0.0506	3.9419	0.0001

Women results - N=574, $R^2=0.3033$, $R^2_{adj}=0.2984$

	coef	se(HC1)	t	p> t
Intercept	-0.0854	0.1595	-0.5353	0.5924
educ	0.1369	0.0101	13.5113	0.0000
exp	0.0529	0.0059	8.9736	0.0000
exp2	-0.0010	0.0001	-6.8237	0.0000
married	-0.0399	0.0407	-0.9813	0.3264

We also calculate and display the turning points for each gender group as follows:

```
In [6]: turn_men = ols_men.params["exp"] / (-2 * ols_men.params["exp2"])
turn_women = ols_women.params["exp"] / (-2 * ols_women.params["exp2"])
print(f"Turning point (men): {turn_men:.2f} years")
print(f"Turning point (women): {turn_women:.2f} years")
```

Turning point (men): 29.78 years

Turning point (women): 26.94 years

Q2. Separate Models for Men and Women

We now estimate the following specification separately by gender:

$$\ln w_i = \beta_0 + \beta_1 \text{educ}_i + \beta_2 \text{exp}_i + \beta_3 \text{exp}_i^2 + \beta_4 \text{married}_i + u_i$$

using heteroskedasticity-robust (HC1) standard errors.

Variable	Coefficient (Men)	Coefficient (Women)
Intercept	0.5197	-0.0854
Education (β_1)	0.1027	0.1369
Experience (β_2)	0.0588	0.0529
Experience ² (β_3)	-0.0010	-0.0010
Married (β_4)	0.1993	-0.0399
N	500	574
R ²	0.2553	0.3033
R ² _adj	0.2493	0.2984

Interpretation.

The education coefficient remains positive and statistically significant for both genders, indicating that additional schooling increases log wages. The estimated effect is somewhat larger for women ($\beta_1 = 0.137$) than for men ($\beta_1 = 0.103$), suggesting that women experience slightly higher percentage returns to education conditional on being in the labor market.

Experience and experience-squared retain their expected signs: wages rise with experience at a diminishing rate. The concave wage–experience profiles imply turning points at approximately:

$$\exp_{men}^* = 29.8 \text{ years}, \quad \exp_{women}^* = 26.9 \text{ years}$$

That is, wage growth tends to level off around 30 years of experience for men and slightly earlier for women.

The marriage coefficient differs sharply by gender. For men, being married is associated with approximately a 19.9% higher wage, statistically significant at the 1% level, consistent with the male marriage premium.

For women, however, the coefficient on *married* is small, negative (-0.040), and statistically insignificant, implying no wage premium (and possibly a mild penalty) for married women.

Finally, the model fits slightly better for women ($R^2 = 0.303$) than for men ($R^2 = 0.255$), indicating that the included covariates explain a bit more of the variation in women's wages. Overall, the gender-specific regressions reinforce patterns commonly observed in labor economics: higher returns to education for women, but a wage premium from marriage primarily accruing to men.

3. Add mean family marriage rate

Construct the mean family marriage rate for each person in the data set (i.e., the fraction of the twins that is married, which can be 0, 1/2, or 1).

```
In [7]: # --- Q3(a): Construct mean family marriage rate ---
twins = twins.copy()

# Mean family marriage rate = average 'married' per famid
twins["mean_married"] = twins.groupby("famid")["married"].transform("mean")

# Check distinct possible values (should be 0, 0.5, 1)
print("Unique family marriage rates:", twins["mean_married"].unique())

# Quick sanity table: family id, t, married, mean_married
display(twins[["famid", "t", "married", "mean_married"]].head(8))
```

Unique family marriage rates: [0. 0.5 1.]

	famid	t	married	mean_married
0	2	1	0	0.0000
1	2	2	0	0.0000
2	4	1	0	0.5000
3	4	2	1	0.5000
4	6	1	1	1.0000
5	6	2	1	1.0000
6	7	1	0	0.0000
7	7	2	0	0.0000

(a)

Verify that when you regress marriage of a twin on the average fraction of the siblings who are married, you get a coefficient of 1.

```
In [8]: # --- Verify auxiliary regression: married_i ~ mean_married_fam ---
import statsmodels.formula.api as smf

aux_reg = smf.ols("married ~ mean_married", data=twins).fit(cov_type="HC1")

tbl_aux = (
    pd.DataFrame({
        "coef": aux_reg.params,
        "se(HC1)": aux_reg.bse,
        "t": aux_reg.tvalues,
        "p>|t|": aux_reg.pvalues
    })
    .round(4)
)

display(tbl_aux)
print(f"N = {int(aux_reg.nobs)}, R^2 = {aux_reg.rsquared:.4f}, R^2_adj = {au
```

	coef	se(HC1)	t	p> t
Intercept	-0.0000	0.0074	-0.0000	1.0000
mean_married	1.0000	0.0009	1,147.5137	0.0000

N = 1074, R² = 0.7370, R²_adj = 0.7367

Q3(a) Constructing the Mean Family Marriage Rate and Verifying the Auxiliary Regression

We define the mean family marriage rate for each family f as the average of the twins' marriage indicators:

$$\overline{\text{married}}_f = \frac{1}{2} \sum_{i=1}^2 \text{married}_{fi},$$

which takes values 0, $\frac{1}{2}$, or 1, depending on whether neither, one, or both twins are married.

We then estimated the auxiliary regression: $\text{married}_{fi} = \alpha + \rho \overline{\text{married}}_f + \varepsilon_{fi}$.

Variable	Coefficient	Robust SE (HC1)	t	p > t
Intercept	-0.0000	0.0074	-0.0000	1.0000
Mean Married ($\overline{\text{married}}_f$)	1.0000	0.0009	1147.51	0.0000

N = 1074 R² = 0.737 R²_adj = 0.737

Interpretation.

The coefficient on the family mean marriage rate is essentially 1, confirming the theoretical result proven in Lecture 8.

When a binary variable is regressed on its own group mean (for groups of equal size), the estimated slope must equal 1 because the group mean perfectly summarizes within-group variation in the dependent variable (I think--I am still very shaky on this idea).

Intuitively, a twin's marriage status is mechanically linked to the family's mean marriage rate—if one twin's marital status changes, the family mean changes in the same direction by exactly half, producing a one-to-one relationship in the regression.

(b)

Add mean fraction of siblings married to your gender-specific wage models from part 2. How does the addition of this variable affect the estimated coefficient on marriage. Give an interpretation of the patterns and how they differ between men and women.

```
In [9]: # --- Q3(b): Add mean_married to the gender-specific models (HC1 robust) ---

import statsmodels.formula.api as smf
import pandas as pd
import numpy as np

# If part-2 objects aren't in memory, quickly recreate them
formula_q2 = "lw ~ educ + exp + exp2 + married"
try:
    ols_men, ols_women
except NameError:
    men = twins.loc[twins["female"] == 0].copy()
    women = twins.loc[twins["female"] == 1].copy()
    ols_men = smf.ols(formula_q2, data=men).fit(cov_type="HC1")
    ols_women = smf.ols(formula_q2, data=women).fit(cov_type="HC1")
```

```
# New models WITH the family mean marriage rate
formula_q3b = "lw ~ educ + exp + exp2 + married + mean_married"

men      = twins.loc[twins["female"] == 0].copy()
women    = twins.loc[twins["female"] == 1].copy()

ols_men_b  = smf.ols(formula_q3b, data=men).fit(cov_type="HC1")
ols_women_b = smf.ols(formula_q3b, data=women).fit(cov_type="HC1")

def tidy(res):
    return (pd.DataFrame({
        "coef": res.params,
        "se(HC1)": res.bse,
        "t": res.tvalues,
        "p>|t|": res.pvalues
    }).round(4))

print(f"Men - N={int(ols_men_b.nobs)}, R²={ols_men_b.rsquared:.4f}, R²_adj={ols_men_b.rsquared_adj:.4f}")
display(tidy(ols_men_b).loc[["Intercept", "educ", "exp", "exp2", "married", "mean_married"]])

print(f"\nWomen - N={int(ols_women_b.nobs)}, R²={ols_women_b.rsquared:.4f}, R²_adj={ols_women_b.rsquared_adj:.4f}")
display(tidy(ols_women_b).loc[["Intercept", "educ", "exp", "exp2", "married", "mean_married"]])
```

Men - N=500, R²=0.2619, R²_adj=0.2544

	coef	se(HC1)	t	p> t
Intercept	0.5102	0.2200	2.3187	0.0204
educ	0.1029	0.0135	7.6062	0.0000
exp	0.0572	0.0079	7.1902	0.0000
exp2	-0.0010	0.0002	-5.8788	0.0000
married	0.0300	0.0897	0.3339	0.7385
mean_married	0.2500	0.1108	2.2571	0.0240

Women - N=574, R²=0.3034, R²_adj=0.2972

	coef	se(HC1)	t	p> t
Intercept	-0.0826	0.1597	-0.5173	0.6050
educ	0.1368	0.0102	13.4580	0.0000
exp	0.0534	0.0062	8.6071	0.0000
exp2	-0.0010	0.0001	-6.6896	0.0000
married	-0.0221	0.0728	-0.3031	0.7618
mean_married	-0.0268	0.0915	-0.2930	0.7695

In [10]: # --- Compare 'married' coefficient before (Q2) vs after adding mean_married

```
def coef(res, name):
    return float(res.params.get(name, np.nan))
```

```

cmp = pd.DataFrame({
    "group": ["Men", "Women"],
    "married_Q2": [coef(ols_men, "married"), coef(ols_women, "married")],
    "married_Q3b": [coef(ols_men_b, "married"), coef(ols_women_b, "married")],
    "mean_married_coef": [coef(ols_men_b, "mean_married"), coef(ols_women_b,
    "mean_married")]
})
cmp["Δ married (Q3b - Q2)"] = (cmp["married_Q3b"] - cmp["married_Q2"])
display(cmp.round(4))

```

	group	married_Q2	married_Q3b	mean_married_coef	Δ married (Q3b - Q2)
0	Men	0.1993	0.0300	0.2500	-0.1693
1	Women	-0.0399	-0.0221	-0.0268	0.0179

Here's a clean Markdown cell you can paste right under your Q3(b) code output. It uses inline math and a tiny comparison table.

Q3(b) Adding the family mean marriage rate to gender-specific wage models

We augment the Q2 models to include the family mean marriage rate $\overline{\text{married}}_f$ and re-estimate $\text{lw}_i = \beta_0 + \beta_1 \text{educ}_i + \beta_2 \text{exp}_i + \beta_3 \text{exp}_i^2 + \beta_4 \text{married}_i + \beta_5 \overline{\text{married}}_f + u_i$, separately for men and women (HC1 SEs).

Key coefficients (marriage terms only):

Group	married_i	$\hat{\beta}_4$ on married_i (Q2)	$\hat{\beta}_4$ on $\overline{\text{married}}_f$ (Q3b)	$\hat{\beta}_5$ on $\overline{\text{married}}_f$ (Q3b)	Δ married (Q3b - Q2)
Men		0.1993	0.0300	0.2500	-0.1693
Women		-0.0399	-0.0221	-0.0268	0.0179

What changed?

- **Men.** The individual marriage premium shrinks from ≈ 0.199 to ≈ 0.030 and loses significance, while the family mean term is positive and significant (≈ 0.25).
- **Women.** The individual marriage coefficient was already small and insignificant (≈ -0.040) and remains small and insignificant after adding the family mean; the family mean term is also small and insignificant.

Interpretation.

- With two-twin families, $\overline{\text{married}}_f = (\text{married}_i + \text{married}_j)/2$. Including $\overline{\text{married}}_f$ absorbs the **family-level (shared) component** of marriage—selection or

factors common to both twins (values, timing, labor supply norms, etc.).

- For **men**, most of the apparent marriage premium in Q2 is captured by this shared component (strong $\hat{\beta}_5 > 0$), leaving little **individual-specific** premium once family mean is controlled. This pattern is consistent with positive selection into marriage among men rather than a large individual causal effect.
- For **women**, there is no evident premium to begin with, and controlling for family mean changes nothing—both the individual and family components are economically small and statistically insignificant.

Model fit. R^2 barely moves for women ($\approx 0.303 \rightarrow 0.303$) and rises slightly for men ($\approx 0.255 \rightarrow 0.262$), consistent with the family mean having explanatory power mainly in the male sample.

(c)

Instead of controlling for the mean family marriage rate, estimate the gender-specific wage models from part 2 with a control for the other twin's marriage status (variable "omarried"). Compare the regression coefficients on (married, omarried) with the coefficient on (married, mean married) in (b).

```
In [11]: # --- Q3(c): Add 'omarried' (other twin married) instead of family mean; con

import statsmodels.formula.api as smf
import pandas as pd
import numpy as np

# If Q2/Q3b results aren't in memory, (re)create them quickly so comparisons
formula_q2 = "lw ~ educ + exp + exp2 + married"
men = twins.loc[twins["female"] == 0].copy()
women = twins.loc[twins["female"] == 1].copy()

try:
    _ = ols_men, ols_women # Q2
except NameError:
    ols_men = smf.ols(formula_q2, data=men).fit(cov_type="HC1")
    ols_women = smf.ols(formula_q2, data=women).fit(cov_type="HC1")

try:
    _ = ols_men_b, ols_women_b # Q3(b): had 'mean_married'
except NameError:
    # Make sure mean_married exists
    if "mean_married" not in twins.columns:
        twins["mean_married"] = twins.groupby("famid")["married"].transform(
            lambda x: x - x.mean()
        )
    men = twins.loc[twins["female"] == 0].copy()
    women = twins.loc[twins["female"] == 1].copy()
    formula_q3b = "lw ~ educ + exp + exp2 + married + mean_married"
    ols_men_b = smf.ols(formula_q3b, data=men).fit(cov_type="HC1")
    ols_women_b = smf.ols(formula_q3b, data=women).fit(cov_type="HC1")

# ---- New Q3(c) models: add 'omarried' (other twin's marriage) ----
```

```

formula_q3c = "lw ~ educ + exp + exp2 + married + omarried"
ols_men_c = smf.ols(formula_q3c, data=men).fit(cov_type="HC1")
ols_women_c = smf.ols(formula_q3c, data=women).fit(cov_type="HC1")

def tidy(res):
    return (pd.DataFrame({
        "coef": res.params,
        "se(HC1)": res.bse,
        "t": res.tvalues,
        "p>|t|": res.pvalues
    })).round(4))

print(f"Men - N={int(ols_men_c.nobs)}, R²={ols_men_c.rsquared:.4f}, R²_adj=
display(tidy(ols_men_c).loc[["Intercept", "educ", "exp", "exp2", "married", "omarried"]])

print(f"\nWomen- N={int(ols_women_c.nobs)}, R²={ols_women_c.rsquared:.4f}, R²_adj=
display(tidy(ols_women_c).loc[["Intercept", "educ", "exp", "exp2", "married", "omarried"]])

# ---- Side-by-side comparison with Q3(b) (mean_married) and Q2 (baseline) ----
def coef(res, name): # safe getter
    return float(res.params.get(name, np.nan))

cmp_c = pd.DataFrame({
    "group": ["Men", "Women"],
    # Q2 (baseline, by sex)
    "married_Q2": [coef(ols_men, "married"), coef(ols_women, "married")],
    # Q3(b) (with mean_married)
    "married_Q3b": [coef(ols_men_b, "married"), coef(ols_women_b, "married")],
    "mean_married_Q3b": [coef(ols_men_b, "mean_married"), coef(ols_women_b, "mean_married")],
    # Q3(c) (with omarried)
    "married_Q3c": [coef(ols_men_c, "married"), coef(ols_women_c, "married")],
    "omarried_Q3c": [coef(ols_men_c, "omarried"), coef(ols_women_c, "omarried")],
})

# Helpful deltas
cmp_c["Δ married (Q3c - Q2)"] = (cmp_c["married_Q3c"] - cmp_c["married_Q2"])
cmp_c["Δ married (Q3c - Q3b)"] = (cmp_c["married_Q3c"] - cmp_c["married_Q3b"])

display(cmp_c.round(4))

```

Men - N=500, R²=0.2619, R²_adj=0.2544

	coef	se(HC1)	t	p> t
Intercept	0.5102	0.2200	2.3187	0.0204
educ	0.1029	0.0135	7.6062	0.0000
exp	0.0572	0.0079	7.1902	0.0000
exp2	-0.0010	0.0002	-5.8788	0.0000
married	0.1550	0.0539	2.8771	0.0040
omarried	0.1250	0.0554	2.2571	0.0240

Women- N=574, R²=0.3034, R²_adj=0.2972

	coef	se(HC1)	t	p> t
Intercept	-0.0826	0.1597	-0.5173	0.6050
educ	0.1368	0.0102	13.4580	0.0000
exp	0.0534	0.0062	8.6071	0.0000
exp2	-0.0010	0.0001	-6.6896	0.0000
married	-0.0355	0.0432	-0.8202	0.4121
omarried	-0.0134	0.0457	-0.2930	0.7695

	group	married_Q2	married_Q3b	mean_married_Q3b	married_Q3c	omarried_Q3c
0	Men	0.1993	0.0300	0.2500	0.1550	0.1250
1	Women	-0.0399	-0.0221	-0.0268	-0.0355	-0.0134

Q3(c) Comparing the “Other Twin Married” and “Mean Married” Models

Replacing the family mean marriage rate $\overline{\text{married}}_f$ with the other twin's marriage status omarried_i , we estimate:

$$\ln w_i = \beta_0 + \beta_1 \text{educ}_i + \beta_2 \text{exp}_i + \beta_3 \text{exp}_i^2 + \beta_4 \text{married}_i + \beta_5 \text{omarried}_i + u_i$$

For men, the coefficient on own marriage increases to 0.155 while the co-twin's marriage coefficient is 0.125. Together, their combined effect ($0.155 + 0.125 \approx 0.28$) is roughly equal to the family mean coefficient (0.25) from part (b). This makes sense since $\overline{\text{married}}_f = \frac{1}{2}(\text{married}_i + \text{omarried}_i)$, meaning the “mean married” variable in (b) effectively averages the two twins' statuses. For women, both coefficients remain small and statistically insignificant, similar to before.

Overall, controlling for the co-twin's marriage shows that the male marriage premium mainly reflects family or shared factors rather than purely individual effects, while for women, marriage continues to have no meaningful association with wages.

4. Three ways to get the within estimator:

In lecture 8 we show three ways to get the within estimator - de-meaned relative to the mean, fixed effects, and control function. Let's run 3 regressions for **male** twins.

(a)

De-mean variables relative to the mean in each family: replace x_{it} by $x_{it} - \bar{x}_i$ where x_{it} includes education, experience-squared, and married status, and replace y_{it} by $y_{it} - \bar{y}_i$. Fit a regression of $(y_{it} - \bar{y}_i)$ on $x_{it} - \bar{x}_i$.

- Note we dropped experience from the regression. See how experience is defined.
The demeaned experience = - demeaned education (correlation = -1).

```
In [ ]: keys = ['famid','t'] # twins data unique at (family, twin) level
select= ['lw','educ','exp','exp2','married']
# focus on families with male twins:
fam_means = twins.loc[twins['female']==0].groupby(['famid'])[select].mean()
fam_means.columns=['famid'] + [f'mean_{x}' for x in select]

# merge fam_means with twins (note you may have defined mean marriage rate as a
male_twins = pd.merge(twins.loc[twins['female']==0], keys+select, fam_means,

# compute the demeaned x and y:
for z in select:
    male_twins[f'd_{z}'] = male_twins[z] - male_twins[f'mean_{z}']

# note: d_exp = - d_educ. exp is defined as age - educ - 6. Within each fami
print(male_twins[['d_exp','d_educ']].corr())
assert (abs(male_twins['d_exp'] + male_twins['d_educ'])<1e-12).all()
```

```
      d_exp  d_educ
d_exp    1.0000 -1.0000
d_educ  -1.0000  1.0000
```

```
In [13]: # OLS for within: formula = "d_lw ~ d_educ + d_exp + d_exp2 + d_married"

# --- Q4(a): Within estimator via family demeaning (MEN only) ---

import pandas as pd
import statsmodels.formula.api as smf

# 1) keep only male twins
men = twins.loc[twins["female"] == 0].copy()

# 2-3) family means and demeaned variables
cols = ["lw", "educ", "exp", "exp2", "married"]
for z in cols:
    men[f"mean_{z}"] = men.groupby("famid")[z].transform("mean")
    men[f"d_{z}"] = men[z] - men[f"mean_{z}"]

# sanity check from the lecture note: d_exp should be - d_educ (corr ≈ -1)
print(men[["d_exp", "d_educ"]].corr().round(3))

# 4) within regression of (y - ybar) on (x - xbar), no intercept
# Drop d_exp due to perfect collinearity with d_educ
formula_w = "d_lw ~ 0 + d_educ + d_exp2 + d_married"
res_w = smf.ols(formula_w, data=men).fit(cov_type="HC1")

# pretty table
tbl_w = (pd.DataFrame({
```



```

    "coef": res_w.params,
    "se(HC1)": res_w.bse,
    "t": res_w.tvalues,
    "p>|t|": res_w.pvalues
}).round(4))

display(tbl_w)
print(f"N = {int(res_w.nobs)}, R² = {res_w.rsquared:.4f}")

```

	d_exp	d_educ		
d_exp	1.0000	-1.0000		
d_educ	-1.0000	1.0000		
	coef	se(HC1)	t	p> t
d_educ	0.0851	0.0369	2.3087	0.0210
d_exp2	0.0004	0.0010	0.4402	0.6598
d_married	0.0247	0.0470	0.5261	0.5988

N = 500, R² = 0.0357

Q4(a) Within Estimator via Family Demeaning

We estimate the within-family regression by demeaning variables for each male twin pair—replacing y_{it} and each regressor x_{it} with deviations from their family means ($y_{it} - \bar{y}_f$) and ($x_{it} - \bar{x}_f$). Since both twins are the same age, experience and education move in opposite directions, producing a correlation of -1 , so we omit d_{exp} from the regression.

The estimated model is:

$d\ln w_i = 0.085 \text{ deduc}_i + 0.0004 \text{ dexp2}_i + 0.025 \text{ dmarried}_i$, with $R^2 = 0.036$ based on 500 male twins.

The within estimator captures *pure* within-family variation—differences between twins holding constant all shared family background factors. The small positive coefficient on $d\text{educ}$ (≈ 0.085) implies that, within a twin pair, the more educated twin earns about 8.5% higher log wages than his sibling. The coefficients on experience squared and marital status are small and insignificant, suggesting little additional within-pair wage difference from these factors once shared family effects are removed.

(b) Fixed Effects

Fit a regression of y_{it} on x_{it} and family fixed effects.

- Note if you use PanelOLS from linearmodels, make sure to drop collinear variables before adding “entityeffects”. For example, educ and exp are collinear with each other conditional on family fixed effects (twins have the same age). We have dropped exp from the regression.

```
In [14]: # --- Q4(b): Fixed effects (MEN only): y_it on x_it + family FE ---
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

men = twins.loc[twins["female"] == 0].copy()
assert {"famid", "t", "lw", "educ", "exp2", "married"} <= set(men.columns)

def tidy_res(params, bse, tvals, pvals):
    return (pd.DataFrame({
        "coef": params, "se(HC1)": bse, "t": tvals, "p>|t|": pvals
    }).round(4))

try:
    # Preferred: linearmodels.PanelOLS (entity fixed effects)
    from linearmodels.panel import PanelOLS
    men_fe = men.set_index(["famid", "t"])
    exog = men_fe[["educ", "exp2", "married"]] # drop exp (collinear with FE)
    mod = PanelOLS(men_fe["lw"], exog, entity_effects=True)
    res = mod.fit(cov_type="robust") # heteroskedasticity-robust

    tbl = tidy_res(res.params, res.std_errors, res.tstats, res.pvalues)
    display(tbl)
    print(f"N = {int(res.nobs)}, R2_within = {res.rsquared_within:.4f}, "
          f"R2_overall = {res.rsquared_overall:.4f}")

except Exception as e:
    # Fallback: OLS with family dummies (same estimand as FE)
    # NB: no 'exp' term, as instructed
    formula_fe = "lw ~ educ + exp2 + married + C(famid)"
    res = smf.ols(formula_fe, data=men).fit(cov_type="HC1")
    # keep only the slope coefficients we care about
    keep = ["educ", "exp2", "married"]
    tbl = tidy_res(res.params[keep], res.bse[keep], res.tvalues[keep], res.pvalues[keep])
    display(tbl)
    print(f"N = {int(res.nobs)}, R2 = {res.rsquared:.4f} (OLS with family c
```

	coef	se(HC1)	t	p> t
educ	0.0851	0.0523	1.6276	0.1049
exp2	0.0004	0.0014	0.3103	0.7566
married	0.0247	0.0666	0.3709	0.7110

N = 500, R2_within = 0.0357, R2_overall = 0.7490

Q4(b) Fixed Effects

We now estimate the wage equation for male twins using a **family fixed effects** model, which controls for all unobserved, time-invariant family characteristics shared by twins (e.g., parental background, neighborhood). The regression is

$lw_{it} = \alpha_f + \beta_1 educ_{it} + \beta_2 exp2_{it} + \beta_3 married_{it} + u_{it}$, where α_f denotes the family fixed effect. Because twins are the same age, $educ$ and exp are perfectly collinear conditional on α_f , so we omit exp from the regression.

The estimated coefficients are $\beta_{educ} = 0.085$, $\beta_{exp2} = 0.0004$, and $\beta_{married} = 0.025$ with $R^2_{within} = 0.036$. These results are nearly identical to those from the within-family (demeaned) regression in part (a), confirming that both approaches yield the same within estimator. The small positive coefficient on education again implies that, holding family background constant, the twin with more education earns about 8.5% higher log wages. The marriage and experience-squared effects remain small and statistically insignificant, suggesting that wage differences within twin pairs are largely explained by education rather than marital status or experience.

(c) Control Function

Fit a regression of y_{it} on x_{it} and means $\bar{x}_i$ in each family. Verify if the coefficients on x_{it} are the same as in (a) and (b).

```
In [15]: # --- Q4(c): Control function (MEN only): y_it on x_it and family means x_i

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

# keep only male twins
men = twins.loc[twins["female"] == 0].copy()

# family means (bar x_i) for the included regressors
for z in ["educ", "exp2", "married"]:
    men[f"m_{z}"] = men.groupby("famid")[z].transform("mean")

# OLS with x_it and family means x_i (no 'exp' term, as in parts (a) and (b))
# HCl robust SEs to stay consistent with earlier parts
formula_c = "lw ~ educ + exp2 + married + m_educ + m_exp2 + m_married"
res_c = smf.ols(formula_c, data=men).fit(cov_type="HC1")

# tidy table of coefficients
tbl_c = (
    pd.DataFrame({
        "coef": res_c.params,
        "se(HC1)": res_c.bse,
        "t": res_c.tvalues,
        "p>|t|": res_c.pvalues
    })
    .loc[["educ", "exp2", "married", "m_educ", "m_exp2", "m_married"]]
    .round(4)
)
display(tbl_c)
print(f"N = {int(res_c.nobs)}, R2 = {res_c.rsquared:.4f}")

# ---- Verify equality of slope coefficients on x_it with 4(a) and 4(b) ----
```

```
# (Assumes you still have `res_w` from 4(a) and `res` from 4(b) in memory.)

def b(res, name):
    return float(res.params.get(name, np.nan))

cmp = pd.DataFrame({
    "coef_4a_demean": [b(res_w, "d_educ"), b(res_w, "d_exp2"), b(res_w, "d_married")],
    "coef_4b_FE": [b(res, "educ"), b(res, "exp2"), b(res, "married")],
    "coef_4c_ctrl": [b(res_c, "educ"), b(res_c, "exp2"), b(res_c, "married")],
    index=["educ", "exp2", "married"]).round(4)

cmp["4c≈4a?"] = np.isclose(cmp["coef_4c_ctrl"], cmp["coef_4a_demean"], atol=1e-4)
cmp["4c≈4b?"] = np.isclose(cmp["coef_4c_ctrl"], cmp["coef_4b_FE"], atol=1e-4)
display(cmp)
```

	coef	se(HC1)	t	p> t
educ	0.0851	0.0689	1.2352	0.2168
exp2	0.0004	0.0018	0.2366	0.8130
married	0.0247	0.0987	0.2504	0.8023
m_educ	0.0030	0.0700	0.0431	0.9656
m_exp2	-0.0003	0.0018	-0.1426	0.8866
m_married	0.3590	0.1199	2.9934	0.0028

N = 500, R2 = 0.1631

	coef_4a_demean	coef_4b_FE	coef_4c_ctrl	4c≈4a?	4c≈4b?
educ	0.0851	0.0851	0.0851	True	True
exp2	0.0004	0.0004	0.0004	True	True
married	0.0247	0.0247	0.0247	True	True

Q4(c) Control Function Interpretation

In this step, I regressed individual log wages y_{it} on both individual characteristics $x_{it} = (\text{educ}, \text{exp}^2, \text{married})$ and the family-level means $\bar{x}_i$ to recover the within-family (control function) estimator. This approach controls for unobserved family-specific heterogeneity by explicitly including the family averages of each regressor rather than demeaning the data or using fixed effects.

The results show that the coefficients on the individual-level regressors—education ($\beta_{\text{educ}} = 0.0851$), experience-squared ($\beta_{\text{exp}^2} = 0.0004$), and marriage ($\beta_{\text{married}} = 0.0247$)—are **identical** to those obtained from the within-family (demeaning) and fixed effects regressions in parts (a) and (b). This confirms the theoretical equivalence of all three estimators: demeaning, fixed effects, and control function all isolate within-family variation to identify the same slope parameters.

However, the inclusion of family means introduces potential multicollinearity since each individual's x_{it} is linearly related to its family mean $\bar{x}_i$. While the results are numerically identical, the control-function approach is less efficient computationally and can be harder to interpret when family-level regressors are strongly correlated with individual deviations. In essence, this exercise illustrates that all three methods capture the same underlying within-family wage relationships but differ in implementation and susceptibility to collinearity or scaling issues.
